

Exp. No. 36

Write a LEX program to identify and count positive and negative numbers.

Program: (positive_neg_nums.l)

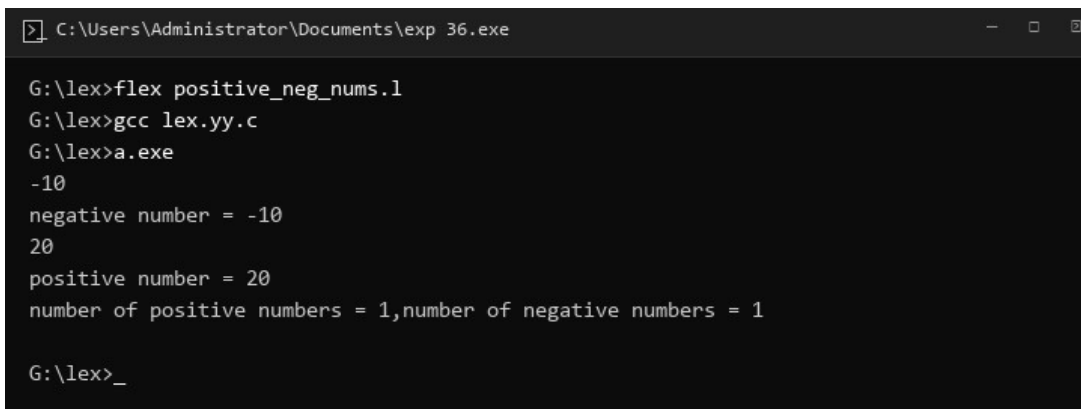
```
%{
int positive_no = 0, negative_no = 0;
}%
%%
^-[0-9]+ {negative_no++;
          printf("negative number = %s\n",
                yytext);} // negative number

[0-9]+ {positive_no++;
        printf("positive number = %s\n",
              yytext);} // positive number

%%
int
yywrap(){}
int main()
{
    yylex();
    printf ("number of positive numbers = %d,"
            "number of negative numbers = %d\n",
            positive_no, negative_no);

    return 0;
}
```

Output:



```
C:\Users\Administrator\Documents\exp 36.exe

G:\lex>flex positive_neg_nums.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
-10
negative number = -10
20
positive number = 20
number of positive numbers = 1,number of negative numbers = 1

G:\lex>_
```

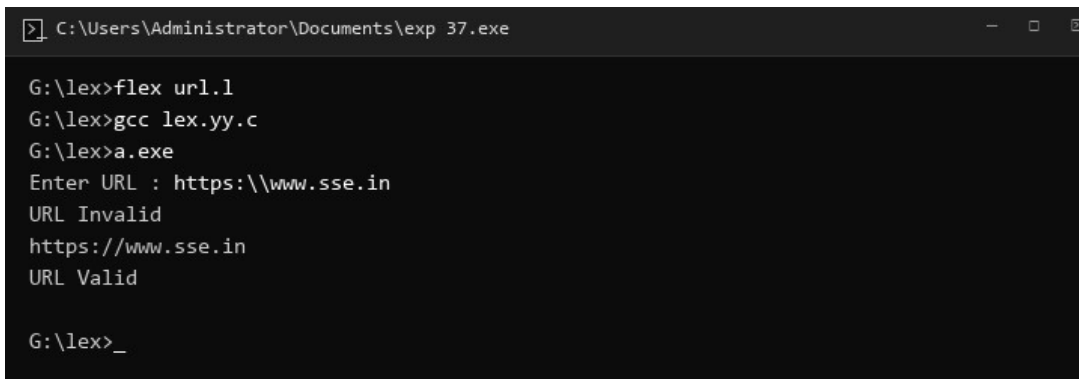
Exp. No. 37

Write a LEX program to validate the URL.

Program: (url.l)

```
%%  
((http)|(ftp))s?:\\[a-zA-Z0-9](.[a-z])+([a-zA-Z0-9+=?]*)* {printf("\nURL Valid\n");}  
  
.+ {printf("\nURL Invalid\n");}  
  
%%  
void main()  
{  
    printf("\nEnter URL :  
"); yylex();  
    printf("\n");  
}  
int yywrap()  
{  
}
```

Output:



```
C:\Users\Administrator\Documents\exp 37.exe  
  
G:\lex>flex url.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
Enter URL : https:\\www.sse.in  
URL Invalid  
https://www.sse.in  
URL Valid  
  
G:\lex>_
```

Exp. No. 38

Write a LEX program to validate DOB of students.

Program: (dob.l)

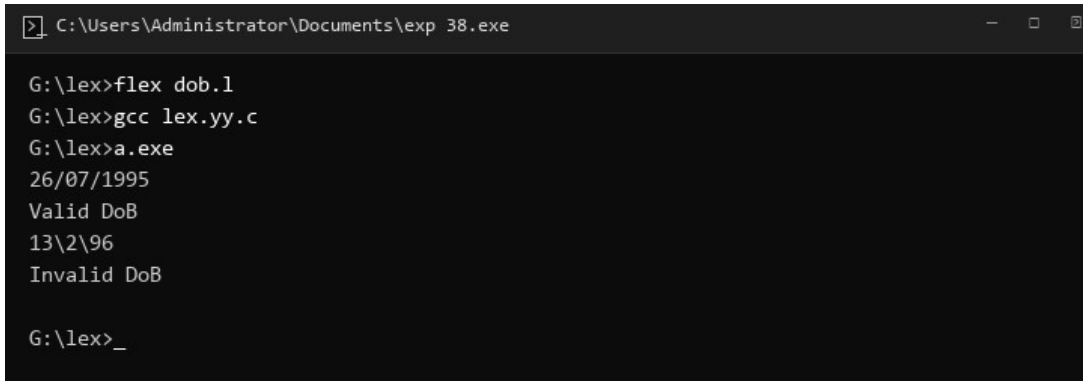
```
%%  
(((0[1-9])|([1-2][0-9])|(3[0-1]))\\((0[1-9])|(1[0-2]))\\(19[0-9]{2}|2[0-9]{3}))  
printf("Valid DoB");  
.* printf("Invalid DoB");  
%%
```

```

int main()
{
    yylex();
    return 0;
}
int yywrap()
{}

```

Output:



```

C:\Users\Administrator\Documents\exp 38.exe

G:\lex>flex dob.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
26/07/1995
Valid DoB
13\2\96
Invalid DoB

G:\lex>_

```

Exp. No. 39

Write a LEX program to check whether the given input is digit or not.

Program: (digit_or_not.l)

```

%%
[0-9]+ {printf("\nValid digit \n");}
.* {printf("\nInvalid digit\n");}
%%
int
yywrap(){}
int main()
{
    yylex();
    return 0;
}

```

Output:

```
C:\Users\Administrator\Documents\exp 39.exe

G:\lex>flex digit_or_not.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
23
Valid digit
h56
Invalid digit

G:\lex>_
```

Exp. No. 40

Write a LEX program to implement basic mathematical operations.

Program: (cal.l)

```
%{
#undef yywrap
#define yywrap() 1
int f1=0,f2=0;
char oper;
float op1=0,op2=0,ans=0;
void eval();
}%

DIGIT [0-9]
NUM
{DIGIT}+(\.{DIGIT}+)?
OP [*/+ -]

%%

{NUM} {
    if(f1==0)
    {
        op1=atof(yytext);
        f1=1;
    }

    else if(f2==1)
    {
        op2=atof(yytext);
        f2=1;
    }
}
```

```
if((f1==1) && (f2==1))
{
    eval();
    f1=0;
    f2=0;
}
}
```

```

{OP} {

    oper=(char) *yytext;
    f2=-1;
}

[\n] {

    if(f1==1 && f2==1)
    {
        eval;
        f1=0;
        f2=0;
    }
}

%%

int main()
{
    yylex();
}

void eval()
{
    switch(oper)
    {
        case '+':
            ans=op1+op2;
            break;

        case '-':
            ans=op1-op2;
            break;

        case '*':
            ans=op1*op2;

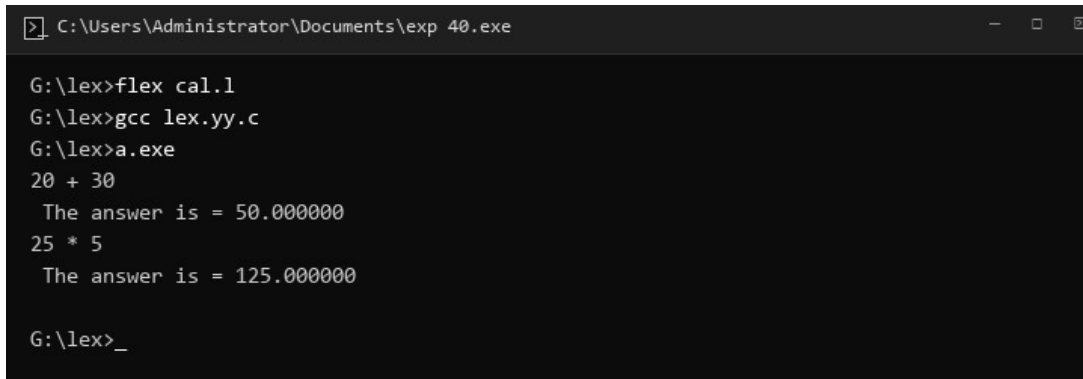
```

```
        break;

case '/':
    if(op2==0)
    {
        printf("ERROR");
        return;
    }
    else
    {
        ans=op1/op2
    }
    break;
default:
```

```
printf("operation not available"); break;
    }
    printf("The answer is = %lf",ans);
}
```

Output:



```
C:\Users\Administrator\Documents\exp 40.exe

G:\lex>flex cal.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
20 + 30
    The answer is = 50.000000
25 * 5
    The answer is = 125.000000

G:\lex>_
```